รายงานสรุปผลการทดลอง

การสร้างและทดสอบ REST API สำหรับระบบจองห้องพัก (Hotel Booking System)

1. วัตถุประสงค์ของการทดลอง

การทดลองในรายวิชาการออกแบบและพัฒนาซอฟต์แวร์ (Software Design and Development) ครั้งนี้ มีวัตถุประสงค์หลักเพื่อศึกษาและฝึกปฏิบัติการพัฒนาระบบให้บริการส่วนหลัง (Backend Service) ด้วยสถาปัตยกรรม REST API ร่วมกับการจัดการฐานข้อมูลแบบเชิงสัมพันธ์ และการทดสอบการทำงานของคำสั่งต่าง ๆ ผ่านโปรแกรมประยุกต์ภายนอก โดยมีขอบเขตการดำเนินงานครอบคลุม 4 มิติหลัก ดังนี้:

- **การจัดการ Version Control**: ฝึกฝนการทำคัดลอกโปรเจกต์ต้นแบบ (Forking) จาก GitHub Repository หลักเข้ามายังบัญชีส่วนตัว และการดึงซอร์สโค้ดลงสู่เครื่องคอมพิวเตอร์ส่วนบุคคล (Git Cloning) เพื่อจัดเตรียมสภาพแวดล้อมการทำงาน
- **การพัฒนาส่วนบริการหลังบ้าน (Backend API)**: การเขียนโปรแกรมด้วยภาษา JavaScript บนสภาพแวดล้อม Node.js ร่วมกับ Express Framework เพื่อสร้าง Endpoint รองรับการรับส่งข้อมูลตามมาตรฐาน HTTP Protocol
- **การจัดการฐานข้อมูล (Database Integration)**: การเชื่อมต่อเซิร์ฟเวอร์เข้ากับฐานข้อมูล SQLite เพื่อจัดเก็บและจัดการข้อมูลการจองห้องพักอย่างเป็นระบบ
- **การทดสอบ API แบบแมนนวล (Manual API Testing)**: การใช้โปรแกรม Postman ในการส่ง Request รูปแบบต่าง ๆ เพื่อตรวจสอบความถูกต้องของระบบและรหัสสถานะการตอบกลับ (HTTP Response Status Codes)

2. สถาปัตยกรรมระบบและโครงสร้างโปรเจกต์

โครงสร้างของระบบจองห้องพักประกอบด้วยการทำงานร่วมกันระหว่างโปรแกรมประยุกต์ Node.js และฐานข้อมูล SQLite3 โดยมีการแบ่งสัดส่วนการทำงานออกเป็นไฟล์หลักตามหลักการออกแบบซอฟต์แวร์ที่ดี ดังนี้:

โครงสร้างไฟล์และโมดูลสำคัญ

- **package.json**: ไฟล์กำหนดค่าคุณลักษณะของโปรเจกต์และรายการเครื่องมือภายนอก (Dependencies) ที่จำเป็น ได้แก่ Express, SQLite3, CORS และ Nodemon สำหรับรันเซิร์ฟเวอร์อัตโนมัติขณะพัฒนา
- **database.js**: โมดูลสำหรับจัดการการเชื่อมต่อฐานข้อมูล bookings.db และสร้างตารางข้อมูลชื่อ bookings หากยังไม่มีในระบบ
- **server.js**: ไฟล์หลักของแอปพลิเคชัน ทำหน้าที่เปิดฟังการเชื่อมต่อบนพอร์ต 3000 และกำหนดเส้นทาง (Routes) การประมวลผลคำสั่ง API ทั้งหมด

โครงสร้างตารางข้อมูล (Database Schema)

ตาราง bookings ในฐานข้อมูล SQLite ถูกออกแบบให้รองรับการเก็บข้อมูลการจองห้องพัก ดังโครงสร้างต่อไปนี้:

ชื่อฟิลด์ (Column Name)	ประเภทข้อมูล (Data Type)	ข้อกำหนด (Constraint)	คำอธิบาย
id	INTEGER	PRIMARY KEY AUTOINCREMENT	รหัสประจำรายการจอง (สร้างให้อัตโนมัติ)
fullname	TEXT	NOT NULL	ชื่อ-นามสกุล ของผู้ใช้บริการ
email	TEXT	NOT NULL	อีเมลติดต่อ
phone	TEXT	NOT NULL	หมายเลขโทรศัพท์
checkin	TEXT	NOT NULL	วันที่เข้าพัก (Check-in)
checkout	TEXT	NOT NULL	วันที่คืนห้องพัก (Check-out)
roomtype	TEXT	NOT NULL	ประเภทห้องพัก (เช่น Standard, Deluxe)
guests	INTEGER	NOT NULL	จำนวนผู้เข้าพัก
status	TEXT	DEFAULT 'confirmed'	สถานะการจอง
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	วันเวลาที่ทำรายการบันทึก

3. ผลการทดสอบการทำงานของ REST API ผ่าน Postman

การทดสอบการทำงานของระบบกระทำผ่านโปรแกรม Postman โดยทำการทดสอบคำสั่งตามมาตรฐาน HTTP Methods ทั้ง 5 รูปแบบ ซึ่งมีผลการทดสอบดังรายละเอียดต่อไปนี้:

3.1 การสร้างรายการจองใหม่ (POST /api/bookings)

- **วัตถุประสงค์**: บันทึกข้อมูลการจองห้องพักรายการใหม่ลงในฐานข้อมูล
- **การส่งข้อมูล (Request Body - JSON)**:

```
{
  "fullname": "สมชาย ใจดี",
  "email": "somchai@example.com",
  "phone": "0812345678",
  "checkin": "2026-06-01",
  "checkout": "2026-06-03",
  "roomtype": "standard",
  "guests": 2
}
```

- **ผลลัพธ์การทำงาน**: ระบบตอบกลับด้วยสถานะ **201 Created** พร้อมคืนค่าหมายเลข id: 1 และข้อความยืนยันการบันทึกสำเร็จ

3.2 การเรียกดูรายการจองทั้งหมด (GET /api/bookings)

- **วัตถุประสงค์**: ดึงข้อมูลรายการจองห้องพักทั้งหมดที่มีอยู่ในฐานข้อมูล
- **ผลลัพธ์การทำงาน**: ระบบตอบกลับด้วยสถานะ **200 OK** พร้อมส่งกลับชุดข้อมูลรายการจองทั้งหมดในรูปแบบอาร์เรย์ของ JSON

3.3 การเรียกดูรายการจองตามรหัสประจำตัว (GET /api/bookings/:id)

- **วัตถุประสงค์**: ค้นหาและแสดงข้อมูลการจองเฉพาะรายการที่ระบุรหัส (ID)
- **ผลลัพธ์การทำงาน**: ระบบตอบกลับด้วยสถานะ **200 OK** พร้อมส่งกลับข้อมูลของรายการจองที่มี ID ตรงตามที่ระบุใน URL

3.4 การแก้ไขปรับปรุงข้อมูลการจอง (PUT /api/bookings/:id)

- **วัตถุประสงค์**: อัปเดตรายละเอียดข้อมูลการจองเดิมในระบบ
- **การส่งข้อมูล (Request Body - JSON)**: ปรับเปลี่ยนประเภทห้องพักเป็น deluxe และแก้ไขวันที่คืนห้องพักเป็น 2026-06-05
- **ผลลัพธ์การทำงาน**: ระบบตอบกลับด้วยสถานะ **200 OK** พร้อมข้อความยืนยันการอัปเดตข้อมูลสำเร็จ

3.5 การลบรายการจอง (DELETE /api/bookings/:id)

- **วัตถุประสงค์**: ถอนหรือลบข้อมูลการจองออกจากฐานข้อมูลตาม ID ที่ระบุ
- **ผลลัพธ์การทำงาน**: ระบบตอบกลับด้วยสถานะ **204 No Content** เพื่อยืนยันว่าการลบรายการเสร็จสมบูรณ์โดยไม่มีเนื้อหาตอบกลับเพิ่มเติม

4. การวิเคราะห์และแก้ไขปัญหาเทคนิคระหว่างการพัฒนา

ในระหว่างกระบวนการพัฒนาและทดลอง ได้พบปัญหาทางเทคนิคและข้อผิดพลาดของการทำงาน ซึ่งนำไปสู่การเรียนรู้และแก้ไขปัญหาอย่างเป็นระบบ ดังนี้:

กรณีศึกษาที่ 1: ข้อผิดพลาดไวยากรณ์ไฟล์กำหนดค่า (EJSONPARSE)

- **ปัญหาที่พบ**: เซิร์ฟเวอร์ไม่สามารถเปิดใช้งานได้เนื่องจากข้อผิดพลาดในไฟล์ package.json
- **การวิเคราะห์**: เกิดจากการปิดเครื่องหมายปีกกา (}) ซ้ำซ้อนและผิดตำแหน่ง ทำให้โครงสร้างไฟล์ JSON ไม่ถูกต้องตามมาตรฐาน
- **การแก้ไข**: ทำการปรับปรุงโครงสร้าง JSON ให้ถูกต้องตามข้อกำหนดของ Node.js เพื่อให้สามารถอ่านและติดตั้งแพ็กเกจได้ตามปกติ

กรณีศึกษาที่ 2: ข้อผิดพลาดจุดสิ้นสุดคำสั่ง (SyntaxError: Unexpected end of input)

- **ปัญหาที่พบ**: โปรแกรม Nodemon หยุดการทำงานและแจ้งเตือนการชนกันของไวยากรณ์ในไฟล์ server.js
- **การวิเคราะห์**: เกิดจากมีอักขระแปลกปลอมและคราบโค้ดค้างอยู่ที่บรรทัดท้ายสุดของไฟล์ ทำให้คำสั่ง app.listen() ปิดไม่สมบูรณ์
- **การแก้ไข**: ทำการลบส่วนของอักขระส่วนเกินออก และปิดบล็อกฟังก์ชัน app.listen ให้ถูกต้อง

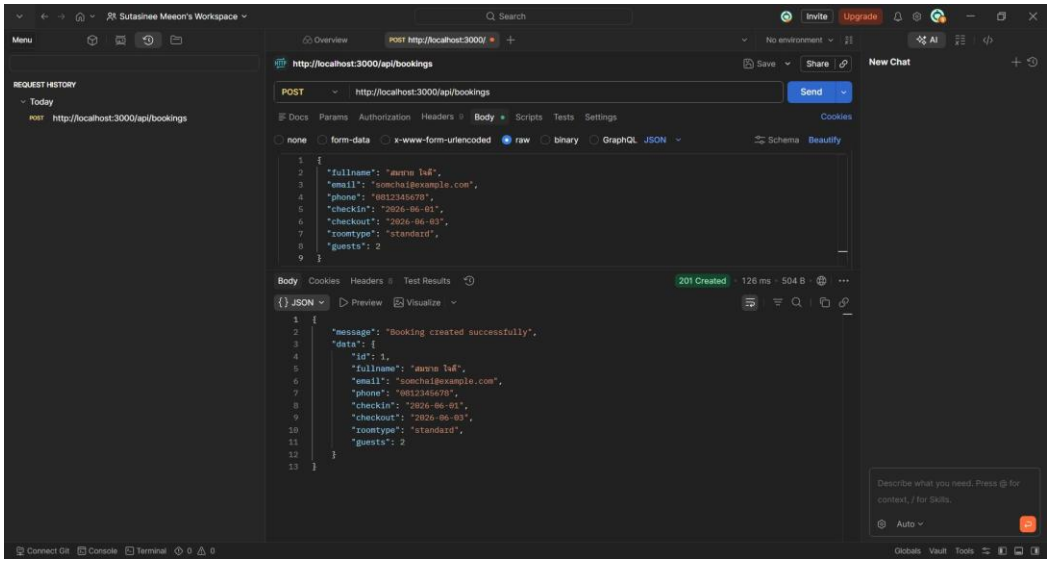
กรณีศึกษาที่ 3: ข้อผิดพลาดการเรียกใช้ฟังก์ชันฐานข้อมูล (500 Internal Server Error)

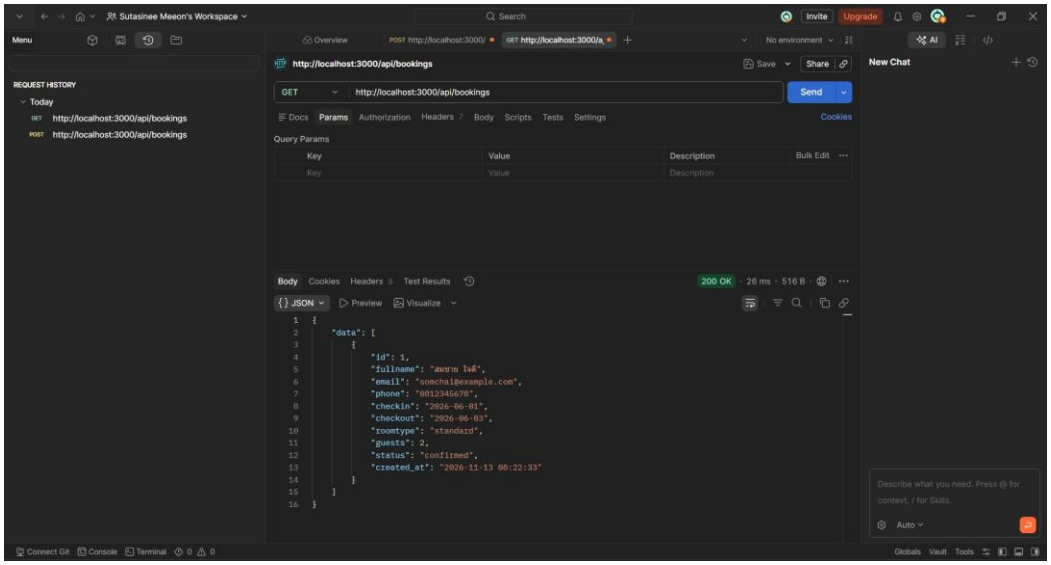
- **ปัญหาที่พบ**: เมื่อยิงคำสั่ง POST จาก Postman ระบบตอบกลับด้วยรหัสผิดพลาด 500 พร้อมระบุ TypeError: db.run is not a function
- **การวิเคราะห์**: เกิดจากการส่งออกอ็อบเจกต์ (Export) ในไฟล์ database.js ไม่สอดคล้องกับการนำเข้า (Import) ในไฟล์ server.js
- **การแก้ไข**: ปรับแก้ไขการใช้ module.exports = db ในไฟล์ database.js ให้ตรงกันกับ const db = require('./database') ในไฟล์ server.js

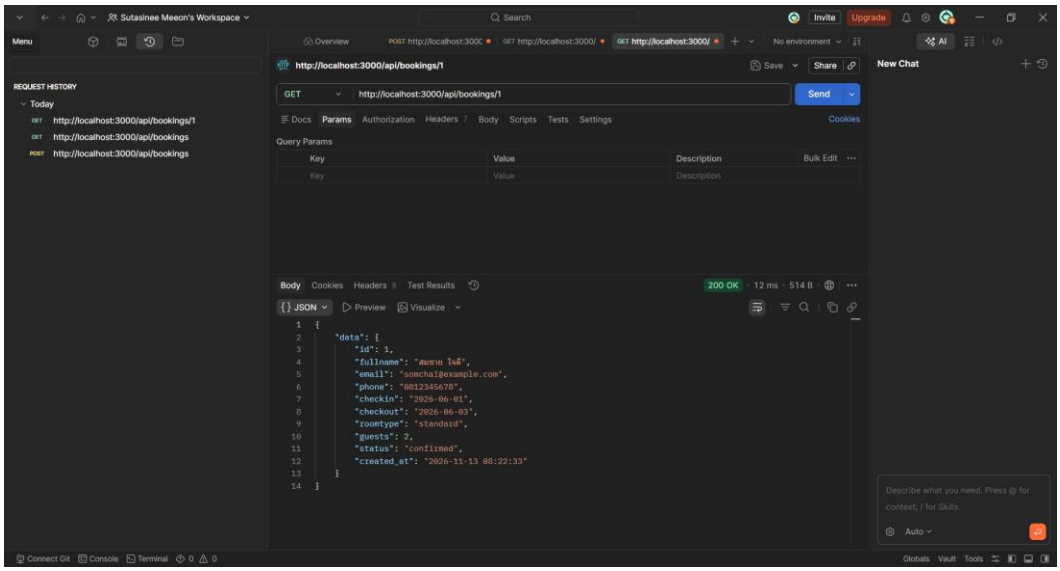
5. สรุปผลการทดลอง

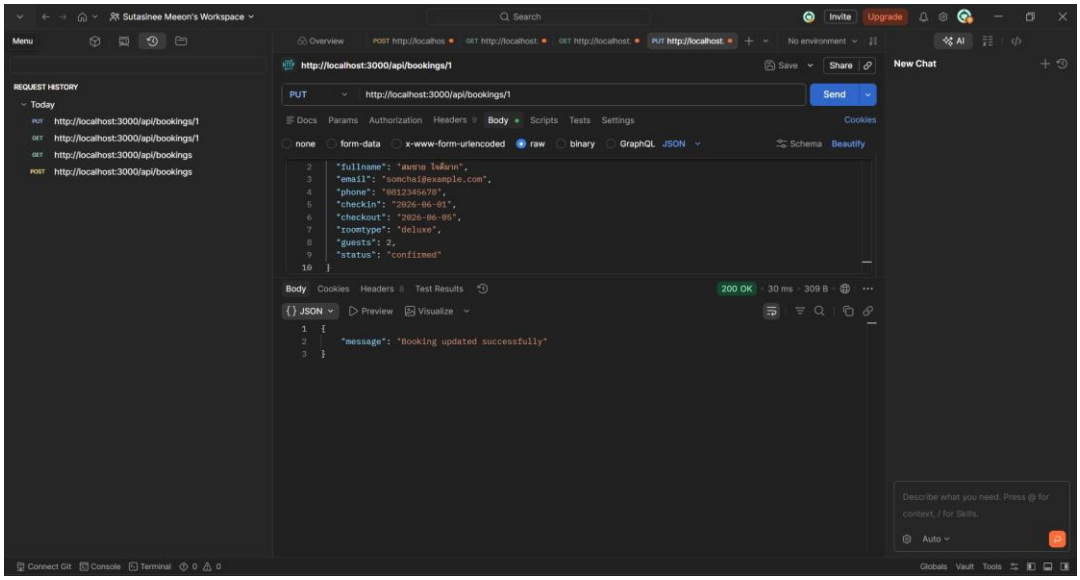
การทดลองพัฒนาระบบ REST API สำหรับบริการจองห้องพักในครั้งนี้ บรรลุผลสำเร็จตามวัตถุประสงค์ที่กำหนดไว้ทุกประการ ผู้ทดลองได้เรียนรู้และเข้าใจกลไกการทำงานของเว็บเซอร์วิสสถาปัตยกรรมแบบ RESTful รวมถึงการจัดเก็บข้อมูลในฐานข้อมูล SQLite และการใช้โปรแกรม Postman ในการตรวจสอบระบบอย่างครบถ้วน

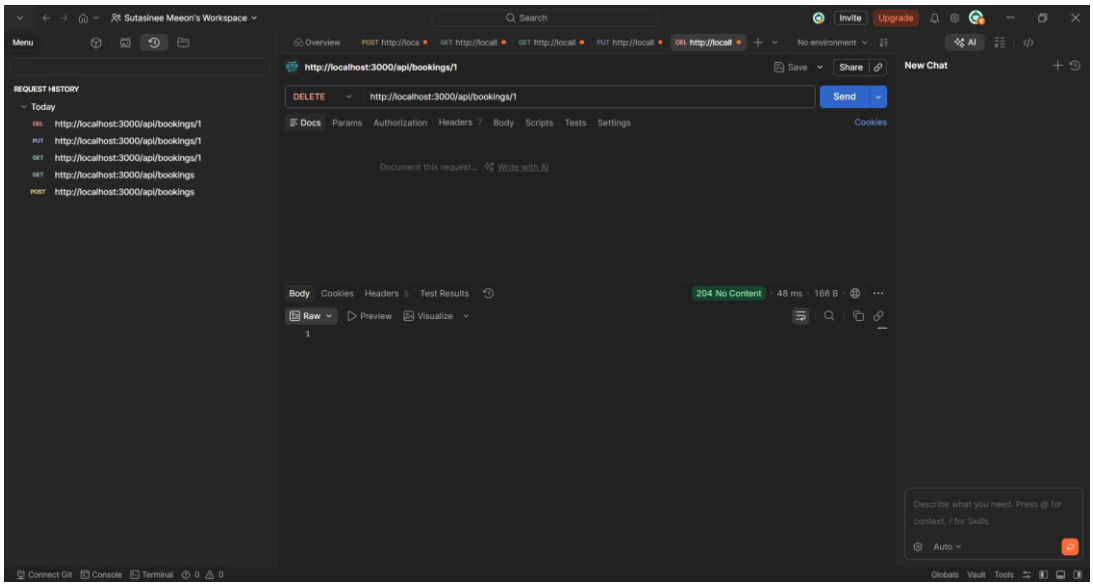
นอกจากนี้ การเผชิญและแก้ไขปัญหาทางเทคนิคด้วยตนเอง เช่น การแก้ไขไฟล์ JSON, การจัดระเบียบโครงสร้างโมดูล Node.js และการไล่หาสาเหตุจาก Error Stack Trace ยังช่วยเสริมสร้างทักษะการวิเคราะห์และแก้ไขปัญหา (Debugging Skills) ซึ่งเป็นพื้นฐานสำคัญยิ่งสำหรับการพัฒนาซอฟต์แวร์ระดับอาชีพต่อไป


Sutasinee Meeon's Workspace
Search
Invite
Upgrade
Menu
Overview
POST http://localhost:3000/
No environment
AI
http://localhost:3000/api/bookings
Save
Share
New Chat
REQUEST HISTORY
Today
POST http://localhost:3000/api/bookings
POST
http://localhost:3000/api/bookings
Send
Docs Params Authorization Headers 9 Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"fullname": "สมชาย ใจดี",
"email": "somchai@example.com",
"phone": "0812345678",
"checkin": "2026-06-01",
"checkout": "2026-06-03",
"roomtype": "standard",
"guests": 2
}
Body Cookies Headers 8 Test Results
201 Created · 126 ms · 504 B
{ } JSON Preview Visualize
{
"message": "Booking created successfully",
"data": [
{
"id": 1,
"fullname": "สมชาย ใจดี",
"email": "somchai@example.com",
"phone": "0812345678",
"checkin": "2026-06-01",
"checkout": "2026-06-03",
"roomtype": "standard",
"guests": 2
}
]
}
Describe what you need. Press @ for context, / for Skills.
Auto
Connect Git
Console
Terminal
Globals
Vault
Tools


Sutasinee Meeon's Workspace
Search
Invite
Upgrade
Menu
Overview
POST http://localhost:3000/
GET http://localhost:3000/a
No environment
AI
http://localhost:3000/api/bookings
Save
Share
New Chat
REQUEST HISTORY
Today
GET http://localhost:3000/api/bookings
POST http://localhost:3000/api/bookings
GET
http://localhost:3000/api/bookings
Send
Docs Params Authorization Headers 7 Body Scripts Tests Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body Cookies Headers 8 Test Results
200 OK · 26 ms · 516 B
{ } JSON Preview Visualize
{
"data": [
{
"id": 1,
"fullname": "สมชาย ใจดี",
"email": "somchai@example.com",
"phone": "0812345678",
"checkin": "2026-06-01",
"checkout": "2026-06-03",
"roomtype": "standard",
"guests": 2,
"status": "confirmed",
"created_at": "2026-11-13 08:22:33"
}
]
}
Describe what you need. Press @ for context, / for Skills.
Auto
Connect Git
Console
Terminal
Globals
Vault
Tools


Sutasinee Meeon's Workspace
Search
Invite
Upgrade
Menu
Overview
GET http://localhost:3000/
No environment
AI
http://localhost:3000/api/bookings/1
Save
Share
New Chat
REQUEST HISTORY
Today
GET http://localhost:3000/api/bookings/1
GET http://localhost:3000/api/bookings
POST http://localhost:3000/api/bookings
GET
http://localhost:3000/api/bookings/1
Send
Docs Params Authorization Headers 7 Body Scripts Tests Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body Cookies Headers 8 Test Results
200 OK · 12 ms · 514 B
{ } JSON Preview Visualize
{
"data": [
{
"id": 1,
"fullname": "สมชาย ใจดี",
"email": "somchai@example.com",
"phone": "0812345678",
"checkin": "2026-06-01",
"checkout": "2026-06-03",
"roomtype": "standard",
"guests": 2,
"status": "confirmed",
"created_at": "2026-11-13 08:22:33"
}
]
}
Describe what you need. Press @ for context, / for Skills.
Auto
Connect Git
Console
Terminal
Globals
Vault
Tools


Sutasinee Meeon's Workspace
Search
Invite
Upgrade
Menu
Overview
No environment
AI
REQUEST HISTORY
Today
PUT http://localhost:3000/api/bookings/1
GET http://localhost:3000/api/bookings/1
GET http://localhost:3000/api/bookings
POST http://localhost:3000/api/bookings
http://localhost:3000/api/bookings/1
Save
Share
New Chat
PUT
http://localhost:3000/api/bookings/1
Send
Docs Params Authorization Headers 9 Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
2 "fullname": "สมชาย ใจดีมาก",
3 "email": "somchai@example.com",
4 "phone": "0812345678",
5 "checkin": "2026-06-01",
6 "checkout": "2026-06-05",
7 "roomtype": "deluxe",
8 "guests": 2,
9 "status": "confirmed"
10 }
Body Cookies Headers 8 Test Results
200 OK · 30 ms · 309 B
{ } JSON Preview Visualize
1 {
2 "message": "Booking updated successfully"
3 }
Describe what you need. Press @ for context, / for Skills.
Auto
Connect Git Console Terminal 0 0
Globals Vault Tools


Sutasinee Meeon's Workspace
Search
Invite
Upgrade
Menu
Overview
No environment
AI
REQUEST HISTORY
Today
DEL http://localhost:3000/api/bookings/1
PUT http://localhost:3000/api/bookings/1
GET http://localhost:3000/api/bookings/1
GET http://localhost:3000/api/bookings
POST http://localhost:3000/api/bookings
http://localhost:3000/api/bookings/1
Save
Share
New Chat
DELETE
http://localhost:3000/api/bookings/1
Send
Docs Params Authorization Headers 7 Body Scripts Tests Settings
Cookies
Document this request... Write with AI
Body Cookies Headers 5 Test Results
204 No Content · 48 ms · 166 B
Raw Preview Visualize
1
Describe what you need. Press @ for context, / for Skills.
Auto
Connect Git Console Terminal 0 0
Globals Vault Tools